



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.I. Edgar Tista García

Asignatura: Estructura de Datos y Algoritmos I

Grupo: 01

No de Práctica(s): 02

Integrante(s): Jiménez Olivo Evelin

*No. de lista o
brigada:* 17

Semestre: 2024-2

Fecha de entrega: 21 de febrero del 2024

Observaciones:

CALIFICACIÓN: _____

Práctica 2. Aplicaciones de Apuntadores

Objetivo:

Utilizar apuntadores en lenguaje C para acceder a las localidades de memoria tanto de datos primitivos como de arreglos.

OBJETIVO DE CLASE:

Utilizar apuntadores para modificar valores de variables indirectamente, así como para realizar el manejo de arreglos y el paso por referencia en funciones

AVANCE

Código (apuntador a carácter)

¿Qué hace el programa?

El código es un programa en el cual se utiliza un apuntador de tipo carácter llamado ap y una variable llamada "c" del mismo tipo. Después se le asigna a la variable c el valor de 'x', luego por medio del operador de dirección & se obtiene la dirección de memoria de la variable c y se guarda en el apuntador ap.

Después por medio de printf el programa imprime 3 cosas:

El primer printf imprime el carácter de la localidad de memoria a la cual apunta.

El segundo printf imprime el código ASCII del carácter almacenado en la dirección de memoria apuntada por 'ap'. Imprime el código ASCII ya que esta haciendo referencia que imprima un entero en una variable de tipo carácter

En el tercer printf imprime la dirección de memoria de la variable c.

```
10 // imprime el carácter de la localidad a la que apunta
11 printf("Carácter: %c\n",*ap);
12 // imprime el código ASCII de la localidad a la que apunta
13 printf("Código ASCII: %d\n",*ap);
14 // imprime la dirección de memoria de la localidad a la que apunta
15 printf("Dirección de memoria: %d\n",ap);
```

¿Cuál es la salida que se muestra en pantalla?

Carácter: x

Código ASCII: 120 //El código ASCII de 'x'

Dirección de memoria: 6487575

Comentario acerca de la relación del ejemplo con el concepto teórico

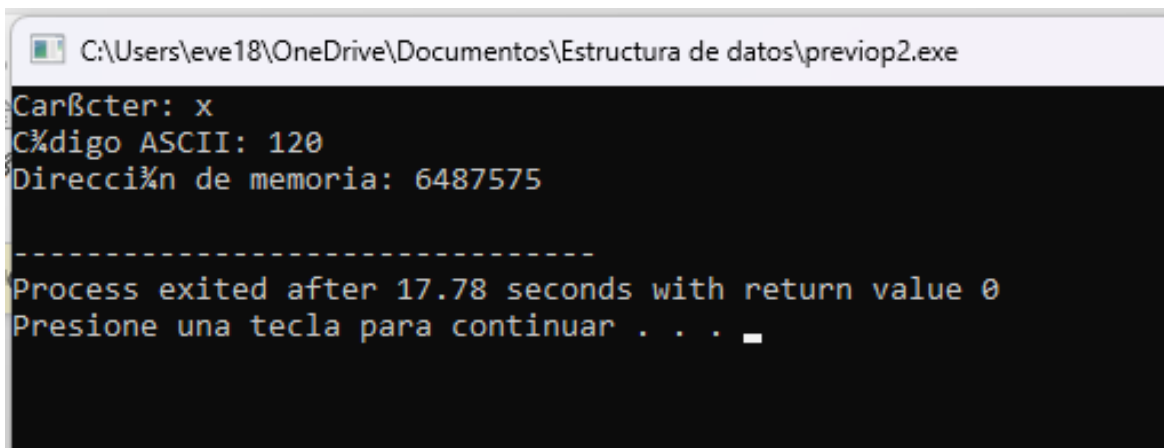
Se utilizan un apuntador llamado 'ap' el cual está asociado a la variable c, esta tiene el valor de 'x'

```
char *ap, c;  
c = 'x';
```

Se usa un operador de dirección (&) para asociar la variable c y así obtener la dirección de memoria de donde se almacena.

```
ap = &c;
```

Implementación



```
C:\Users\eve18\OneDrive\Documentos\Estructura de datos\previop2.exe  
Carácter: x  
Código ASCII: 120  
Dirección de memoria: 6487575  
-----  
Process exited after 17.78 seconds with return value 0  
Presione una tecla para continuar . . . _
```

Código (apuntador a entero)

¿Qué hace el programa?

Primero el código comienza declarando las variables, declara dos variables de tipo short llamadas a y b con los valores iniciales de 5 y 10, respectivamente. Luego, declara un apuntador de tipo short llamado 'apEnt'

Después, al apuntador apEnt por medio de un operador de dirección (&) se obtiene la dirección de memoria de almacenamiento de la variable 'a'.

Luego imprime el valor de a.

En la línea 13 del código se le asigna a la variable b el valor del apuntador 'apEnt', lo que hace que b tenga el valor de a, es decir 5 y en la línea 15 se imprima este valor.

```
13 | b = *apEnt; // b = 5
14 | // imprime el valor de lo que apunta apEnt
15 | printf("b = %i // *apEnt\n", b);
```

Después, se asigna *apEnt + 1 a la variable b, por lo que incrementa en uno el valor al que apunta 'apEnt' dando, así como resultado de b=6 resultando en b. Se imprime este valor.

```
16 | b = *apEnt+1; // b = 6
17 | // imprime el valor de lo que apunta apEnt + 1
18 | printf("b = %i // *apEnt+1\n", b);
```

Luego, se asigna el valor 0 al que apunta apEnt, lo que modifica el valor de 'a' a 0. Se imprime el nuevo valor de a.

¿Cuál es la salida que se muestra en pantalla?

a = 5

b = 5 // *apEnt

b = 6 // *apEnt+1

a = 0 // *apEnt = 0

Comentario acerca de la relación del ejemplo con el concepto teórico

En primer lugar, se usa un operador de dirección (&) para asociarlo a la variable "a" y así obtener la dirección de memoria de donde se almacena.

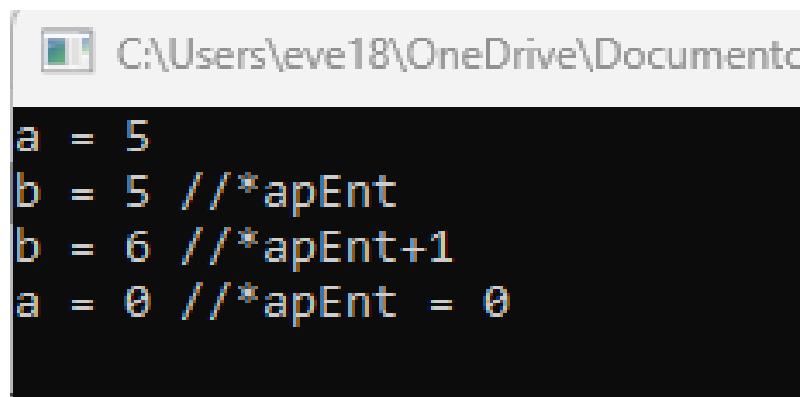
En este ejemplo los apuntadores se utilizan para modificar los valores de la variable 'a'.

Primero la variable a tenía un valor de 5 y después se le asigno un valor de 0 como podemos observar en las imágenes de abajo.

```
7 | {  
8 |     short a = 5, b = 10;  
9 |     short *apEnt;  
10 |     apEnt = &a;  
11 |     // imprime el valor enter
```

```
19 |     *apEnt = 0; // a = 0  
20 |     // le asigna el valor de 0 a la variable al que apunta apEnt  
21 |     printf("a = %i // *apEnt = 0\n", a);  
22 |     return 0;  
23 | }
```

Implementación



```
a = 5  
b = 5 // *apEnt  
b = 6 // *apEnt+1  
a = 0 // *apEnt = 0
```

Código (apuntador a arreglo)

¿Qué hace el programa?

El programa crea un arreglo unidimensional de tipo short llamado 'arr' con tamaño 5 y un apuntador del mismo tipo llamado 'apArr'.

Por medio del operador de dirección (&) se obtiene la dirección de memoria donde se almacena el primer elemento del arreglo 'arr' y se esta se guarda en el apuntador 'apArr'.

Después el programa imprime 3 cosas por medio de printf. En primer lugar, imprime la dirección de memoria donde se encuentra el primer elemento del arreglo 'arr'. Luego se imprime la dirección de memoria del arreglo 'arr'. Aquí el nombre del arreglo es un apuntador a la primera localidad de memoria del arreglo. Por último, imprime la dirección de memoria del apuntador 'apArr'.

```
11 printf("Dirección del arreglo en la primera posición: %x\n",&arr[0]);
12 // imprime la dirección de memoria del arreglo
13 // (el nombre del arreglo es un apuntador)
14 printf("Dirección del arreglo: %x\n",&arr);
15 // imprime la dirección de memoria del apuntador apArr
16 printf("Dirección del apuntador: %x\n",apArr);
```

¿Cuál es la salida que se muestra en pantalla?

Dirección del arreglo en la primera posición: 62fe00

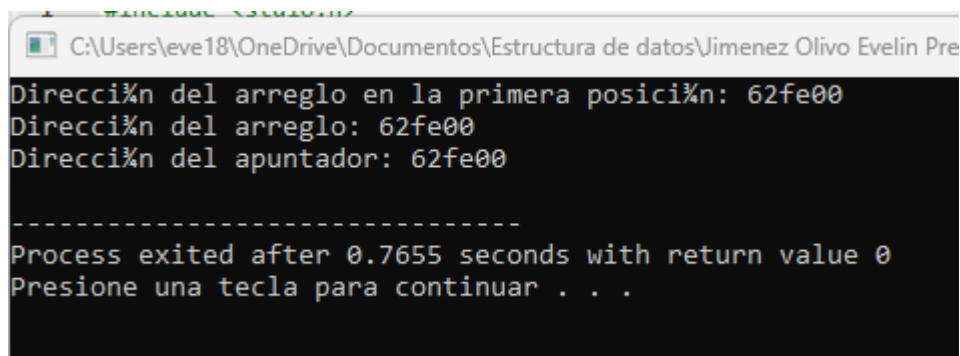
Dirección del arreglo: 62fe00

Dirección del apuntador: 62fe00

Comentario acerca de la relación del ejemplo con el concepto teórico

Utilizan apuntadores para ver la dirección de memoria de donde se encuentra el primer elemento del arreglo 'arr', del arreglo 'arr' y apuntador 'apArr'.

Implementación



```
C:\Users\eve18\OneDrive\Documentos\Estructura de datos\Jimenez Olivo Evelin Pre
Dirección del arreglo en la primera posición: 62fe00
Dirección del arreglo: 62fe00
Dirección del apuntador: 62fe00

-----
Process exited after 0.7655 seconds with return value 0
Presione una tecla para continuar . . .
```

Código (Paso de variables por valor y por referencia)

¿Qué hace el programa?

Es un programa el cual tiene 3 funciones. La función 'pasarValor', la función 'pasarReferencia' y la función principal 'main'

En la función principal (main), se declara un arreglo de enteros nums y se asigna su dirección de memoria al apuntador 'ap'. Luego se imprime el valor apuntado por ap.

Después, se llama a la función pasarValor y se le pasa el valor apuntado por ap. Esta función imprime el valor recibido y luego lo cambia a 128. Sin embargo, esto no afecta al valor original del arreglo nums[0], ya que el valor del parámetro equis se recibe por valor. Esto significa que se pasa una copia del valor original a la función. Luego se imprime nuevamente el valor apuntado por ap para demostrar que no ha cambiado.

```
16 void pasarValor(int equis)
17 {
18     printf("%d\n", equis);
19     equis = 128;
20     printf("%d\n", equis);
21 }
```

Luego, se llama a la función pasarReferencia y se le pasa la dirección de memoria apuntada por ap. Esta función imprime el valor al que apunta y luego lo cambia a 128. Esto sí afecta al valor original de nums[0], ya que se está modificando directamente en la memoria y esto sucede porque el valor del parámetro equis se recibe por referencia, es decir, se pasa la dirección de memoria del valor original a la función. Igualmente se imprime el valor para demostrar que si se modificó.

```
void pasarReferencia(int *equis)
{
    printf("%d\n", *equis);
    *equis = 128;
    printf("%d\n", *equis);
}
```

¿Cuál es la salida que se muestra en pantalla?

Pasar valor: 55

55

128

Pasar referencia: 55

55

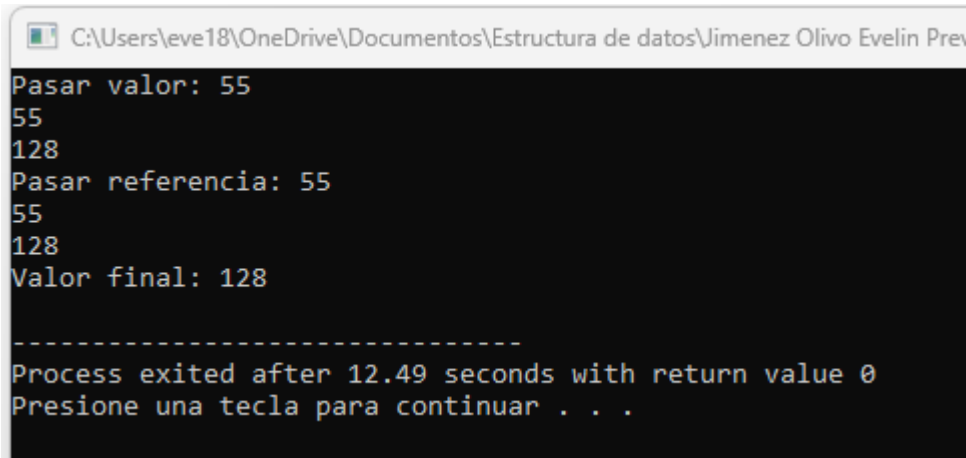
128

Valor final: 128

Comentario acerca de la relación del ejemplo con el concepto teórico

Se utilizan los apuntadores para pasar valores por referencia en la función pasarReferencia(), lo que permite modificar directamente el valor original en la memoria. Mientras que en la función pasarValor(), se pasan valores por valor, lo que implica que se trabaja con copias de los valores originales, sin afectar a los datos originales en la memoria.

Implementación



```
C:\Users\eve18\OneDrive\Documentos\Estructura de datos\Jimenez Olivo Evelin Prev
Pasar valor: 55
55
128
Pasar referencia: 55
55
128
Valor final: 128

-----
Process exited after 12.49 seconds with return value 0
Presione una tecla para continuar . . .
```


Código (aritmética de direcciones)

¿Qué hace el programa?

Primero en el código se declara un arreglo unidimensional de tipo short llamado 'arr' con tamaño de 5 elementos e inicializa sus valores {91, 28, 73, 46, 55}. Igualmente se declara un apuntador llamado 'apArr' a este se le asigna el primer elemento del arreglo 'arr'

Se imprime el primer elemento del arreglo ya que hay un apuntador que apunta al inicio del arreglo.

```
11 // apunta al inicio del arreglo
12 printf("*apArr = %i\n", *apArr);
13 // suma una localidad al índice
```

Después por medio de la aritmética de direcciones 'apArr +1' se suma una localidad al inicio del arreglo y se imprime el segundo valor del arreglo, en este caso el 28.

Se hace lo mismo que en el párrafo anterior, por medio de la aritmética de direcciones 'apArr +2' se suma dos localidades al inicio del arreglo y se imprime el tercer valor del arreglo, en este caso el 73.

```
14 printf("*(apArr+1) = %i\n", *(apArr+1));
15 printf("*(apArr+2) = %i\n", *(apArr+2));
```

Arreglo:

0	1	2	3	4
91	28	73	46	55
				
Línea 12	Línea 14	Línea 15		

¿Cuál es la salida que se muestra en pantalla?

*apArr = 91

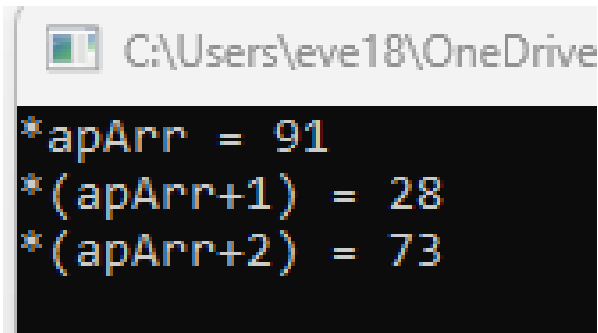
*(apArr+1) = 28

*(apArr+2) = 73

Comentario acerca de la relación del ejemplo con el concepto teórico

Se declara un apuntador llamado 'apArr', para apuntar a las localidades de los arreglos y así por medio de la aritmética de direcciones y el apuntador poder recorrer al arreglo

Implementación



```
*apArr = 91
*(apArr+1) = 28
*(apArr+2) = 73
```

Código (arreglo unidimensional)

¿Qué hace el programa?

Primero se declara un arreglo de tipo short llamado 'nums' e inicializa sus valores

{55, 44, 33, 22, 11}, el cual indica que el tamaño del arreglo es 5

Se declara un apuntador llamado 'ap' y una variable llamada cont.

Se asigna al apuntador 'ap' el arreglo nums.

Después se utiliza un ciclo for para poder iterar sobre el arreglo, utilizando la aritmética de direcciones. El ciclo va desde cont = 0 hasta cont<5.

En cada iteración del ciclo, se imprime el valor hexadecimal del elemento del arreglo en el que se encuentra, después se va recorriendo cada índice del arreglo

```
11 | printf("%x\n", (ap+cont));
```

¿Cuál es la salida que se muestra en pantalla?

62fe00

62fe02

62fe04

62fe06

62fe08

Errores

```
9 | ap = nums;
```

```
9 | ap = &nums;
```

Le falto el operador (&) para obtener asociar el apuntador con la variable

Comentario acerca de la relación del ejemplo con el concepto teórico

Se utilizan apuntadores para ver la dirección de memoria de cada elemento del arreglo 'nums' aunque sabemos que para eso no sirven los apuntadores

Implementación

```
C:\Users\eve18\OneDrive\Documentos\Estructura de datos\Jimenez Olivo Evelin Pre
62fe00
62fe02
62fe04
62fe06
62fe08

-----
Process exited after 0.0554 seconds with return value 0
Presione una tecla para continuar . . .
```

Código (arreglo bidimensional)

¿Qué hace el programa?

Se declara un arreglo dimensional de tipo entero llamado 'nums' e inicializa sus valores y se declaró un apuntador llamado 'ap'.

Se le asigna al apuntador 'ap' el arreglo "nums"

Después se utiliza un ciclo for para poder iterar sobre el arreglo, utilizando la aritmética de apuntadores. El ciclo va desde índice = 0 hasta índice<5.

En cada iteración del ciclo for se imprime el valor hexadecimal del elemento del arreglo en el que se encuentra. Después Esto se va recorriendo cada índice del arreglo.

Pero si el residuo de dividir el índice entre 3 es igual a 0, significa que ya se llenó el renglón, por lo que se imprime un salto de línea para imprimir el siguiente renglón.

```

.2 | for (indice = 0; indice < 9 ; indice++)
.3 | {
.4 |     if ((indice%3)==0)
.5 |         printf("\n");
.6 |         printf("%x\t", (ap+indice));
.7 |     }
.8 |
```

¿Cuál es la salida que se muestra en pantalla?

62fde0 62fde4 62fde8

62fdec 62fdf0 62fdf4

62fdf8 62fdfc 62fe00

Comentario acerca de la relación del ejemplo con el concepto teórico

Se utilizan apuntadores para ver la dirección de memoria de cada elemento del arreglo 'nums'.

Errores

Le falto el operador (&) para obtener la dirección de memoria

```

.1 | ap = nums;
.2 |
.3 |
.4 |
.5 |
.6 |
.7 |
.8 |
.9 |
10 |
11 |
12 |
13 |
14 |
15 |
16 |
17 |
18 |
19 |
20 |
21 |
22 |
23 |
24 |
25 |
26 |
27 |
28 |
29 |
30 |
31 |
32 |
33 |
34 |
35 |
36 |
37 |
38 |
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100 |
101 |
102 |
103 |
104 |
105 |
106 |
107 |
108 |
109 |
110 |
111 |
112 |
113 |
114 |
115 |
116 |
117 |
118 |
119 |
120 |
121 |
122 |
123 |
124 |
125 |
126 |
127 |
128 |
129 |
130 |
131 |
132 |
133 |
134 |
135 |
136 |
137 |
138 |
139 |
140 |
141 |
142 |
143 |
144 |
145 |
146 |
147 |
148 |
149 |
150 |
151 |
152 |
153 |
154 |
155 |
156 |
157 |
158 |
159 |
160 |
161 |
162 |
163 |
164 |
165 |
166 |
167 |
168 |
169 |
170 |
171 |
172 |
173 |
174 |
175 |
176 |
177 |
178 |
179 |
180 |
181 |
182 |
183 |
184 |
185 |
186 |
187 |
188 |
189 |
190 |
191 |
192 |
193 |
194 |
195 |
196 |
197 |
198 |
199 |
200 |
201 |
202 |
203 |
204 |
205 |
206 |
207 |
208 |
209 |
210 |
211 |
212 |
213 |
214 |
215 |
216 |
217 |
218 |
219 |
220 |
221 |
222 |
223 |
224 |
225 |
226 |
227 |
228 |
229 |
230 |
231 |
232 |
233 |
234 |
235 |
236 |
237 |
238 |
239 |
240 |
241 |
242 |
243 |
244 |
245 |
246 |
247 |
248 |
249 |
250 |
251 |
252 |
253 |
254 |
255 |
256 |
257 |
258 |
259 |
260 |
261 |
262 |
263 |
264 |
265 |
266 |
267 |
268 |
269 |
270 |
271 |
272 |
273 |
274 |
275 |
276 |
277 |
278 |
279 |
280 |
281 |
282 |
283 |
284 |
285 |
286 |
287 |
288 |
289 |
290 |
291 |
292 |
293 |
294 |
295 |
296 |
297 |
298 |
299 |
300 |
301 |
302 |
303 |
304 |
305 |
306 |
307 |
308 |
309 |
310 |
311 |
312 |
313 |
314 |
315 |
316 |
317 |
318 |
319 |
320 |
321 |
322 |
323 |
324 |
325 |
326 |
327 |
328 |
329 |
330 |
331 |
332 |
333 |
334 |
335 |
336 |
337 |
338 |
339 |
340 |
341 |
342 |
343 |
344 |
345 |
346 |
347 |
348 |
349 |
350 |
351 |
352 |
353 |
354 |
355 |
356 |
357 |
358 |
359 |
360 |
361 |
362 |
363 |
364 |
365 |
366 |
367 |
368 |
369 |
370 |
371 |
372 |
373 |
374 |
375 |
376 |
377 |
378 |
379 |
380 |
381 |
382 |
383 |
384 |
385 |
386 |
387 |
388 |
389 |
390 |
391 |
392 |
393 |
394 |
395 |
396 |
397 |
398 |
399 |
400 |
401 |
402 |
403 |
404 |
405 |
406 |
407 |
408 |
409 |
410 |
411 |
412 |
413 |
414 |
415 |
416 |
417 |
418 |
419 |
420 |
421 |
422 |
423 |
424 |
425 |
426 |
427 |
428 |
429 |
430 |
431 |
432 |
433 |
434 |
435 |
436 |
437 |
438 |
439 |
440 |
441 |
442 |
443 |
444 |
445 |
446 |
447 |
448 |
449 |
450 |
451 |
452 |
453 |
454 |
455 |
456 |
457 |
458 |
459 |
460 |
461 |
462 |
463 |
464 |
465 |
466 |
467 |
468 |
469 |
470 |
471 |
472 |
473 |
474 |
475 |
476 |
477 |
478 |
479 |
480 |
481 |
482 |
483 |
484 |
485 |
486 |
487 |
488 |
489 |
490 |
491 |
492 |
493 |
494 |
495 |
496 |
497 |
498 |
499 |
500 |
501 |
502 |
503 |
504 |
505 |
506 |
507 |
508 |
509 |
510 |
511 |
512 |
513 |
514 |
515 |
516 |
517 |
518 |
519 |
520 |
521 |
522 |
523 |
524 |
525 |
526 |
527 |
528 |
529 |
530 |
531 |
532 |
533 |
534 |
535 |
536 |
537 |
538 |
539 |
540 |
541 |
542 |
543 |
544 |
545 |
546 |
547 |
548 |
549 |
550 |
551 |
552 |
553 |
554 |
555 |
556 |
557 |
558 |
559 |
560 |
561 |
562 |
563 |
564 |
565 |
566 |
567 |
568 |
569 |
570 |
571 |
572 |
573 |
574 |
575 |
576 |
577 |
578 |
579 |
580 |
581 |
582 |
583 |
584 |
585 |
586 |
587 |
588 |
589 |
590 |
591 |
592 |
593 |
594 |
595 |
596 |
597 |
598 |
599 |
600 |
601 |
602 |
603 |
604 |
605 |
606 |
607 |
608 |
609 |
610 |
611 |
612 |
613 |
614 |
615 |
616 |
617 |
618 |
619 |
620 |
621 |
622 |
623 |
624 |
625 |
626 |
627 |
628 |
629 |
630 |
631 |
632 |
633 |
634 |
635 |
636 |
637 |
638 |
639 |
640 |
641 |
642 |
643 |
644 |
645 |
646 |
647 |
648 |
649 |
650 |
651 |
652 |
653 |
654 |
655 |
656 |
657 |
658 |
659 |
660 |
661 |
662 |
663 |
664 |
665 |
666 |
667 |
668 |
669 |
670 |
671 |
672 |
673 |
674 |
675 |
676 |
677 |
678 |
679 |
680 |
681 |
682 |
683 |
684 |
685 |
686 |
687 |
688 |
689 |
690 |
691 |
692 |
693 |
694 |
695 |
696 |
697 |
698 |
699 |
700 |
701 |
702 |
703 |
704 |
705 |
706 |
707 |
708 |
709 |
710 |
711 |
712 |
713 |
714 |
715 |
716 |
717 |
718 |
719 |
720 |
721 |
722 |
723 |
724 |
725 |
726 |
727 |
728 |
729 |
730 |
731 |
732 |
733 |
734 |
735 |
736 |
737 |
738 |
739 |
740 |
741 |
742 |
743 |
744 |
745 |
746 |
747 |
748 |
749 |
750 |
751 |
752 |
753 |
754 |
755 |
756 |
757 |
758 |
759 |
760 |
761 |
762 |
763 |
764 |
765 |
766 |
767 |
768 |
769 |
770 |
771 |
772 |
773 |
774 |
775 |
776 |
777 |
778 |
779 |
780 |
781 |
782 |
783 |
784 |
785 |
786 |
787 |
788 |
789 |
790 |
791 |
792 |
793 |
794 |
795 |
796 |
797 |
798 |
799 |
800 |
801 |
802 |
803 |
804 |
805 |
806 |
807 |
808 |
809 |
810 |
811 |
812 |
813 |
814 |
815 |
816 |
817 |
818 |
819 |
820 |
821 |
822 |
823 |
824 |
825 |
826 |
827 |
828 |
829 |
830 |
831 |
832 |
833 |
834 |
835 |
836 |
837 |
838 |
839 |
840 |
841 |
842 |
843 |
844 |
845 |
846 |
847 |
848 |
849 |
850 |
851 |
852 |
853 |
854 |
855 |
856 |
857 |
858 |
859 |
860 |
861 |
862 |
863 |
864 |
865 |
866 |
867 |
868 |
869 |
870 |
871 |
872 |
873 |
874 |
875 |
876 |
877 |
878 |
879 |
880 |
881 |
882 |
883 |
884 |
885 |
886 |
887 |
888 |
889 |
890 |
891 |
892 |
893 |
894 |
895 |
896 |
897 |
898 |
899 |
900 |
901 |
902 |
903 |
904 |
905 |
906 |
907 |
908 |
909 |
910 |
911 |
912 |
913 |
914 |
915 |
916 |
917 |
918 |
919 |
920 |
921 |
922 |
923 |
924 |
925 |
926 |
927 |
928 |
929 |
930 |
931 |
932 |
933 |
934 |
935 |
936 |
937 |
938 |
939 |
940 |
941 |
942 |
943 |
944 |
945 |
946 |
947 |
948 |
949 |
950 |
951 |
952 |
953 |
954 |
955 |
956 |
957 |
958 |
959 |
960 |
961 |
962 |
963 |
964 |
965 |
966 |
967 |
968 |
969 |
970 |
971 |
972 |
973 |
974 |
975 |
976 |
977 |
978 |
979 |
980 |
981 |
982 |
983 |
984 |
985 |
986 |
987 |
988 |
989 |
990 |
991 |
992 |
993 |
994 |
995 |
996 |
997 |
998 |
999 |
1000 |
1001 |
1002 |
1003 |
1004 |
1005 |
1006 |
1007 |
1008 |
1009 |
1010 |
1011 |
1012 |
1013 |
1014 |
1015 |
1016 |
1017 |
1018 |
1019 |
1020 |
1021 |
1022 |
1023 |
1024 |
1025 |
1026 |
1027 |
1028 |
1029 |
1030 |
1031 |
1032 |
1033 |
1034 |
1035 |
1036 |
1037 |
1038 |
1039 |
1040 |
1041 |
1042 |
1043 |
1044 |
1045 |
1046 |
1047 |
1048 |
1049 |
1050 |
1051 |
1052 |
1053 |
1054 |
1055 |
1056 |
1057 |
1058 |
1059 |
1060 |
1061 |
1062 |
1063 |
1064 |
1065 |
1066 |
1067 |
1068 |
1069 |
1070 |
1071 |
1072 |
1073 |
1074 |
1075 |
1076 |
1077 |
1078 |
1079 |
1080 |
1081 |
1082 |
1083 |
1084 |
1085 |
1086 |
1087 |
1088 |
1089 |
1090 |
1091 |
1092 |
1093 |
1094 |
1095 |
1096 |
1097 |
1098 |
1099 |
1100 |
1101 |
1102 |
1103 |
1104 |
1105 |
1106 |
1107 |
1108 |
1109 |
1110 |
1111 |
1112 |
1113 |
1114 |
1115 |
1116 |
1117 |
1118 |
1119 |
1120 |
1121 |
1122 |
1123 |
1124 |
1125 |
1126 |
1127 |
1128 |
1129 |
1130 |
1131 |
1132 |
1133 |
1134 |
1135 |
1136 |
1137 |
1138 |
1139 |
1140 |
1141 |
1142 |
1143 |
1144 |
1145 |
1146 |
1147 |
1148 |
1149 |
1150 |
1151 |
1152 |
1153 |
1154 |
1155 |
1156 |
1157 |
1158 |
1159 |
1160 |
1161 |
1162 |
1163 |
1164 |
1165 |
1166 |
1167 |
1168 |
1169 |
1170 |
1171 |
1172 |
1173 |
1174 |
1175 |
1176 |
1177 |
1178 |
1179 |
1180 |
1181 |
1182 |
1183 |
1184 |
1185 |
1186 |
1187 |
1188 |
1189 |
1190 |
1191 |
1192 |
1193 |
1194 |
1195 |
1196 |
1197 |
1198 |
1199 |
1200 |
1201 |
1202 |
1203 |
1204 |
1205 |
1206 |
1207 |
1208 |
1209 |
1210 |
1211 |
1212 |
1213 |
1214 |
1215 |
1216 |
1217 |
1218 |
1219 |
1220 |
1221 |
1222 |
1223 |
1224 |
1225 |
1226 |
1227 |
1228 |
1229 |
1230 |
1231 |
1232 |
1233 |
1234 |
1235 |
1236 |
1237 |
1238 |
1239 |
1240 |
1241 |
1242 |
1243 |
1244 |
1245 |
1246 |
1247 |
1248 |
1249 |
1250 |
1251 |
1252 |
1253 |
1254 |
1255 |
1256 |
1257 |
1258 |
1259 |
1260 |
1261 |
1262 |
1263 |
1264 |
1265 |
1266 |
1267 |
1268 |
1269 |
1270 |
1271 |
1272 |
1273 |
1274 |
1275 |
1276 |
1277 |
1278 |
1279 |
1280 |
1281 |
1282 |
1283 |
1284 |
1285 |
1286 |
1287 |
1288 |
1289 |
1290 |
1291 |
1292 |
1293 |
1294 |
1295 |
1296 |
1297 |
1298 |
1299 |
1300 |
1301 |
1302 |
1303 |
1304 |
1305 |
1306 |
1307 |
1308 |
1309 |
1310 |
1311 |
1312 |
1313 |
1314 |
1315 |
1316 |
1317 |
1318 |
1319 |
1320 |
1321 |
1322 |
1323 |
1324 |
1325 |
1326 |
1327 |
1328 |
1329 |
1330 |
1331 |
1332 |
1333 |
1334 |
1335 |
1336 |
1337 |
1338 |
1339 |
1340 |
1341 |
1342 |
1343 |
1344 |
1345 |
1346 |
1347 |
1348 |
1349 |
1350 |
1351 |
1352 |
1353 |
1354 |
1355 |
1356 |
1357 |
1358 |
1359 |
1360 |
1361 |
1362 |
1363 |
1364 |
1365 |
1366 |
1367 |
1368 |
1369 |
1370 |
1371 |
1372 |
1373 |
1374 |
1375 |
1376 |
1377 |
1378 |
1379 |
1380 |
1381 |
1382 |
1383 |
1384 |
1385 |
1386 |
1387 |
1388 |
1389 |
1390 |
1391 |
1392 |
1393 |
1394 |
1395 |
1396 |
1397 |
1398 |
1399 |
1400 |
1401 |
1402 |
1403 |
1404 |
1405 |
1406 |
1407 |
1408 |
1409 |
1410 |
1411 |
1412 |
1413 |
1414 |
1415 |
1416 |
1417 |
1418 |
1419 |
1420 |
1421 |
1422 |
1423 |
1424 |
1425 |
1426 |
1427 |
1428 |
1429 |
1430 |
1431 |
1432 |
1433 |
1434 |
1435 |
1436 |
1437 |
1438 |
1439 |
1440 |
1441 |
1442 |
1443 |
1444 |
1445 |
1446 |
1447 |
1448 |
1449 |
1450 |
1451 |
1452 |
1453 |
1454 |
1455 |
1456 |
1457 |
1458 |
1459 |
1460 |
1461 |
1462 |
1463 |
1464 |
1465 |
1466 |
1467 |
1468 |
1469 |
1470 |
1471 |
1472 |
1473 |
1474 |
1475 |
1476 |
1477 |
1478 |
1479 |
1480 |
1481 |
1482 |
1483 |
1484 |
1485 |
1486 |
1487 |
1488 |
1489 |
1490 |
1491 |
1492 |
1493 |
1494 |
1495 |
1496 |
1497 |
1498 |
1499 |
1500 |
1501 |
1502 |
1503 |
1504 |
1505 |
1506 |
1507 |
1508 |
1509 |
1510 |
1511 |
1512 |
1513 |
1514 |
1515 |
1516 |
1517 |
1518 |
1519 |
1520 |
1521 |
1522 |
1523 |
1524 |
1525 |
1526 |
1527 |
1528 |
1529 |
1530 |
1531 |
1532 |
1533 |
1534 |
1535 |
1536 |
1537 |
1538 |
1539 |
1540 |
1541 |
1542 |
1543 |
1544 |
1545 |
1546 |
1547 |
1548 |
1549 |
1550 |
1551 |
1552 |
1553 |
1554 |
1555 |
1556 |
1557 |
1558 |
1559 |
1560 |
1561 |
1562 |
1563 |
1564 |
1565 |
1566 |
1567 |
1568 |
1569 |
1570 |
1571 |
1572 |
1573 |
1574 |
1575 |
1576 |
1577 |
1578 |
1579 |
1580 |
1581 |
1582 |
1583 |
1584 |
1585 |
1586 |
1587 |
1588 |
1589 |
1590 |
1591 |
1592 |
1593 |
1594 |
1595 |
1596 |
1597 |
1598 |
1599 |
1600 |
1601 |
1602 |
1603 |
1604 |
1605 |
1606 |
1607 |
1608 |
1609 |
1610 |
1611 |
1612 |
1613 |
1614 |
1615 |
1616 |
1617 |
1618 |
1619 |
1620 |
1621 |
1622 |
1623 |
1624 |
1625 |
1626 |
1627 |
1628 |
1629 |
1630 |
1631 |
1632 |
1633 |
1634 |
1635 |
1636 |
1637 |
1638 |
1639 |
1640 |
1641 |
1642 |
1643 |
1644 |
1645 |
1646 |
1647 |
1648 |
1649 |
1650 |
1651 |
1652 |
1653 |
1654 |
1655 |
1656 |
1657 |
1658 |
1659 |
1660 |
1661 |
1662 |
1663 |
1664 |
1665 |
1666 |
1667 |
1668 |
1669 |
1670 |
1671 |
1672 |
1673 |
1674 |
1675 |
1676 |
1677 |
1678 |
1679 |
1680 |
1681 |
1682 |
1683 |
1684 |
1685 |
1686 |
1687 |
1688 |
1689 |
1690 |
1691 |
1692 |
1693 |
1694 |
1695 |
1696 |
1697 |
1698 |
1699 |
1700 |
1701 |
1702 |
1703 |
1704 |
1705 |
1706 |
1707 |
1708 |
1709 |
1710 |
1711 |
1712 |
1713 |
1714 |
1715 |
1716 |
1717 |
1718 |
1719 |
1720 |
1721 |
1722 |
1723 |
1724 |
1725 |
1726 |
1727 |
1728 |
1729 |
1730 |
1731 |
1732 |
1733 |
1734 |
1735 |
1736 |
1737 |
1738 |
1739 |
1740 |
1741 |
1742 |
1743 |
1744 |
1745 |
1746 |
1747 |
1748 |
1749 |
1750 |
1751 |
1752 |
1753 |
1754 |
1755 |
1756 |
1757 |
1758 |
1759 |
1760 |
1761 |
1762 |
1763 |
1764 |
1765 |
1766 |
1767 |
1768 |
1769 |
1770 |
1771 |
1772 |
1773 |
1774 |
1775 |
1776 |
1777 |
1778 |
1779 |
1780 |
1781 |
1782 |
1783 |
1784 |
1785 |
1786 |
1787 |
1788 |
1789 |
1790 |
1791 |
1792 |
1793 |
1794 |
1795 |
1796 |
1797 |
1798 |
1799 |
1800 |
1801 |
1802 |
1803 |
1804 |
1805 |
1806 |
1807 |
1808 |
1809 |
1810 |
1811 |
1812 |
1813 |
1814 |
1815 |
1816 |
1817 |
1818 |
1819 |
1820 |
1821 |
1822 |
1823 |
1824 |
1825 |
1826 |
1827 |
1828 |
1829 |
1830 |
1831 |
1832 |
1833 |
1834 |
1835 |
1836 |
1837 |
1838 |
1839 |
1840 |
1841 |
1842 |
1843 |
1844 |
1845 |
1846 |
1847 |
1848 |
1849 |
1850 |
1851 |
1852 |
1853 |
1854 |
1855 |
1856 |
1857 |
1858 |
1859 |
1860 |
1861 |
1862 |
1863 |
1864 |
1865 |
1866 |
1867 |
1868 |
1869 |
1870 |
1871 |
1872 |
1873 |
1874 |
1875 |
1876 |
1877 |
1878 |
1879 |
1880 |
1881 |
1882 |
1883 |
1884 |
1885 |
1886 |
1887 |
1888 |
1889 |
1890 |
1891 |
1892 |
1893 |
1894 |
1895 |
1896 |
1897 |
1898 |
1899 |
1900 |
1901 |
1902 |
1903 |
1904 |
1905 |
1906 |
1907 |
1908 |
1909 |
1910 |
1911 |
1912 |
1913 |
1914 |
1915 |
1916 |
1917 |
1918 |
1919 |
1920 |
1921 |
1922 |
1923 |
1924 |
1925 |
1926 |
1927 |
1928 |
1929 |
1930 |
1931 |
1932 |
1933 |
1934 |
1935 |
1936 |
1937 |
1938 |
1939 |
1940 |
1941 |
1942 |
1943 |
1944 |
1945 |
1946 |
1947 |
1948 |
1949 |
1950 |
1951 |
1952 |
1953 |
1954 |
1955 |
1956 |
1957 |
1958 |
1959 |
1960 |
1961 |
1962 |
1963 |
1964 |
1965 |
1966 |
1967 |
1968 |
1969 |
1970 |
1971 |
1972 |
1973 |
1974 |
1975 |
1976 |
1977 |
1978 |
1979 |
1980 |
1981 |
1982 |
1983 |
1984 |
1985 |
1986 |
1987 |
1988 |
1989 |
1990 |
1991 |
1992 |
1993 |
1994 |
1995 |
1996 |
1997 |
1998 |
1999 |
2000 |
2001 |
2002 |
2003 |
2004 |
2005 |
2006 |
2007 |
2008 |
2009 |
2010 |
2011 |
2012 |
2013 |
2014 |
2015 |
2016 |
2017 |
2018 |
2019 |
2020 |
2021 |
2022 |
2023 |
2024 |
2025 |
2026 |
2027 |
2028 |
2029 |
2030 |
2031 |
2032 |
2033 |
2034 |
2035 |
2036 |
2037 |
2038 |
2039 |
2040 |
2041 |
2042 |
2043 |
2044 |
2045 |
2046 |
2047 |
2048 |
2049 |
2050 |
2051 |
2052 |
2053 |
2054 |
2055 |
2056 |
2057 |
2058 |
2059 |
2060 |
2061 |
2062 |
2063 |
2064 |
2065 |
2066 |
2067 |
2068 |
2069 |
2070 |
2071 |
2072 |
2073 |
2074 |
2075 |
2076 |
2077 |
2078 |
2079 |
2080 |
2081 |
2082 |
2083 |
2084 |
2085 |
2086 |
2087 |
2088 |
2089 |
2090 |
2091 |
2092 |
2093 |
20
```

Implementación

```
C:\Users\eve18\OneDrive\Documentos\Estructura de datos\Jimenez Olivo Evelin Previc  
62fde0 62fde4 62fde8  
62fdec 62fdf0 62fdf4  
62fdf8 62fdfc 62fe00  
-----  
Process exited after 13.06 seconds with return value 0  
Presione una tecla para continuar . . .
```

Cifrado de Cesar

¿Qué hace el programa?

Este programa implementa el cifrado César, que es un tipo de cifrado en el cual se desplaza el abecedario 3 posiciones

Primero se definen dos arreglos de caracteres 'abecedarioEnClaro' y 'abecedarioCifrado' donde se inicializan uno con el alfabeto y el otro con el alfabeto cifrado respectivamente.

Se definen dos funciones: cifrar y descifrar. En estas funciones se toma una cadena de caracteres como entrada y posteriormente se aplica el cifrado o descifrado de cesar, respectivamente.

En la función principal "main" se muestra un menú con 3 opciones: cifrar, descifrar o salir para que posteriormente se lea la opción ingresada por el usuario.

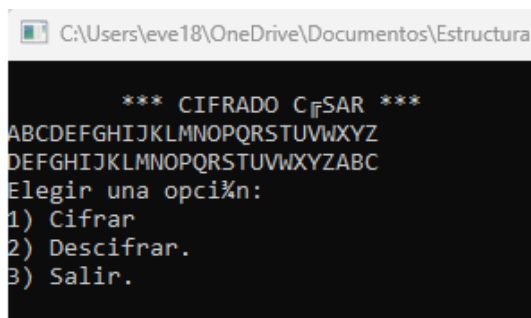
Dependiendo de la opción ingresada al usuario se llama a la función cifrar o a la función descifrar, para que el usuario proporcione una palabra para ser cifrada o descifrada.

Las funciones cifrar y descifrar recorren cada letra de la palabra proporcionada y buscan su equivalente en el alfabeto correspondiente. Luego imprimen la letra equivalente en el alfabeto cifrado o en el alfabeto normal.

Se imprime el menú hasta que el usuario elija la opción salir.

¿Cuál es la salida que se muestra en pantalla?

La salida en pantalla es un menú de 3 opciones las cuales son: cifrar, descifrar, salir. Para que posteriormente el usuario elija la opción que desea.



```
C:\Users\eve18\OneDrive\Documentos\Estructura
*** CIFRADO CÉSAR ***
ABCDEFGHIJKLMNOPQRSTUVWXYZ
DEFGHIJKLMNOPQRSTUVWXYZABC
Elegir una opción:
1) Cifrar
2) Descifrar.
3) Salir.
```

Comentario acerca de la relación del ejemplo con el concepto teórico

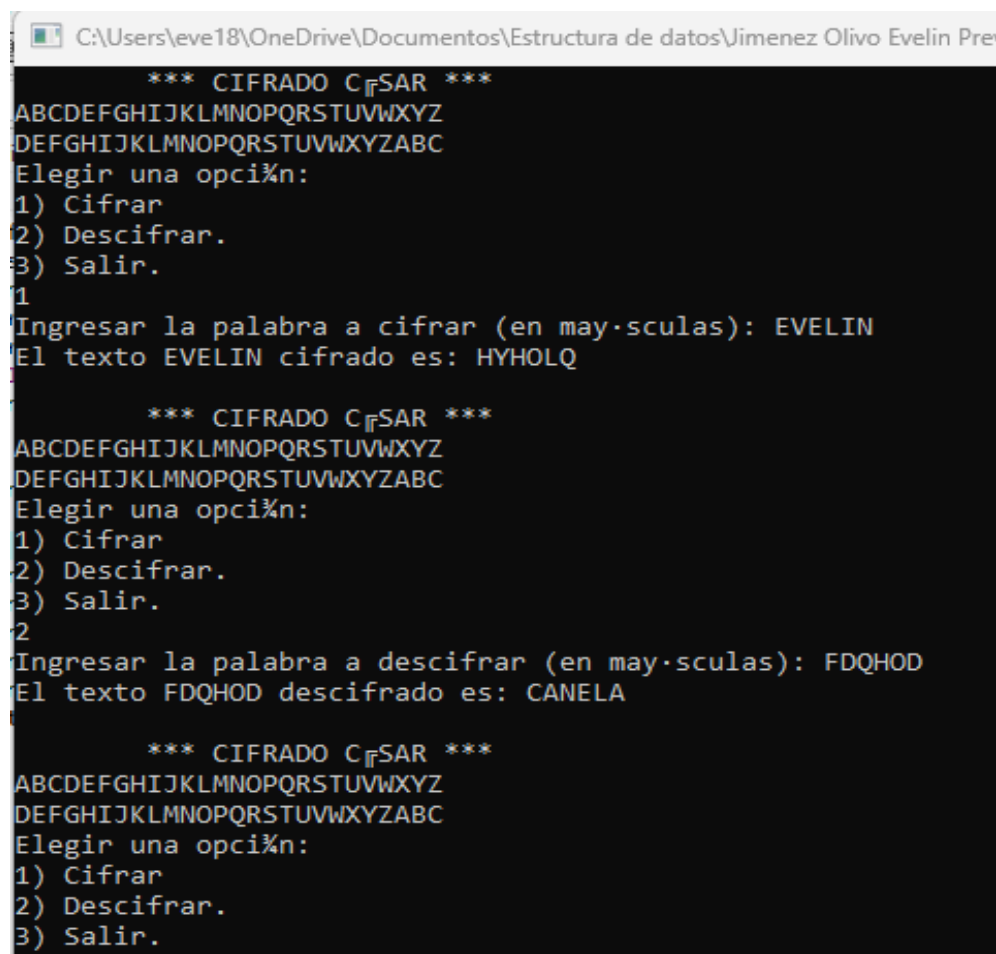
En la función cifrar se utiliza un apuntador llamado 'textoEnClaro', para acceder a cada letra de la palabra que se desea cifrar. Sucede algo similar en la función

descifrar, se utiliza un apuntador llamado 'textoCifrado' para acceder a cada letra de la palabra que se desea descifrar.

En la función principal 'main' se utilizan apuntadores para recorrer los arreglos y para eso se utilizan aritmética de direcciones.

```
20     for (contador=0 ; contador<26; contador++)
21         printf("%c", *(abecedarioEnClaro+contador));
22     printf("\n");
23     for (contador=0 ; contador<26; contador++)
24         printf("%c", *(abecedarioCifrado+contador));
25     printf("\nElegir una opción:\n");
```

Implementación:



```
C:\Users\eve18\OneDrive\Documentos\Estructura de datos\Jimenez Olivo Evelin Pre
*** Cifrado César ***
ABCDEFGHIJKLMNOPQRSTUVWXYZ
DEFGHIJKLMNOPQRSTUVWXYZABC
Elegir una opción:
1) Cifrar
2) Descifrar.
3) Salir.
1
Ingresar la palabra a cifrar (en mayúsculas): EVELIN
El texto EVELIN cifrado es: HYHOLQ

*** Cifrado César ***
ABCDEFGHIJKLMNOPQRSTUVWXYZ
DEFGHIJKLMNOPQRSTUVWXYZABC
Elegir una opción:
1) Cifrar
2) Descifrar.
3) Salir.
2
Ingresar la palabra a descifrar (en mayúsculas): FDQHOD
El texto FDQHOD descifrado es: CANELA

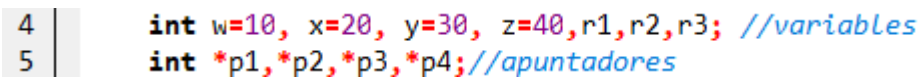
*** Cifrado César ***
ABCDEFGHIJKLMNOPQRSTUVWXYZ
DEFGHIJKLMNOPQRSTUVWXYZABC
Elegir una opción:
1) Cifrar
2) Descifrar.
3) Salir.
```

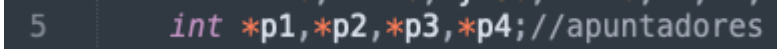
Ejercicio 1

Codifica el siguiente programa

- Explica y corrige todos los errores que encuentras
- Indica que hace y cuál es la salida en pantalla

El ejercicio se declaran 4 variables a las cuales se les asigna un valor de tipo entero, después se declaran otras 4 variables. Posteriormente se declaran apuntadores. Por lo que aquí hay un error ya que originalmente p3 lo declara como variable y no como apuntador

Error: 

Corregido: 

Por lo que corregimos declarando p3 como apuntador, agregándole el operador (*).

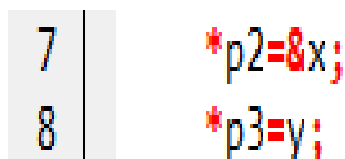
Después asociamos los apuntadores con otras variables por medio del operador de dirección (&), a los apuntadores p1, p2 y p3 se les asocia a las variables w, x y y respectivamente.

En esas 3 líneas de código hay 2 errores, los cuales son:

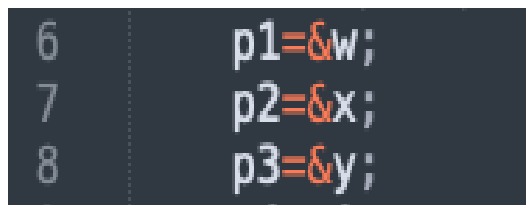
Línea 7: En “*p2=&x;” como se asocia el apuntador con la variable no se le debería colocar el operador (*) ya que, si se le coloca se esta indicando que el apuntador, será un apuntador y eso no tiene congruencia.

Línea 8: En “*p3=y;” sucede un error igual de como se asocia el apuntador con la variable no se le debería colocar el operador (*) y antes de la variable “y” se le debe agregar el operador (&).

Errores:

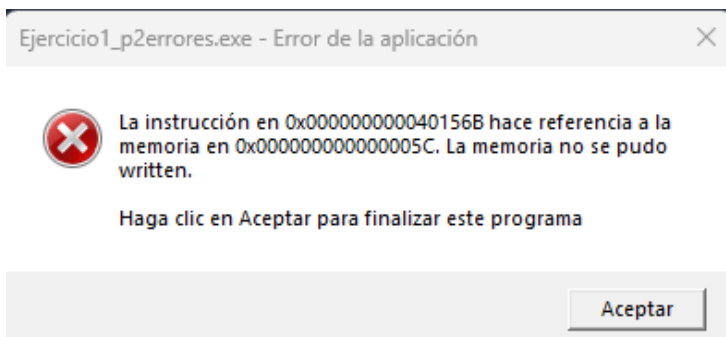


Corregido:

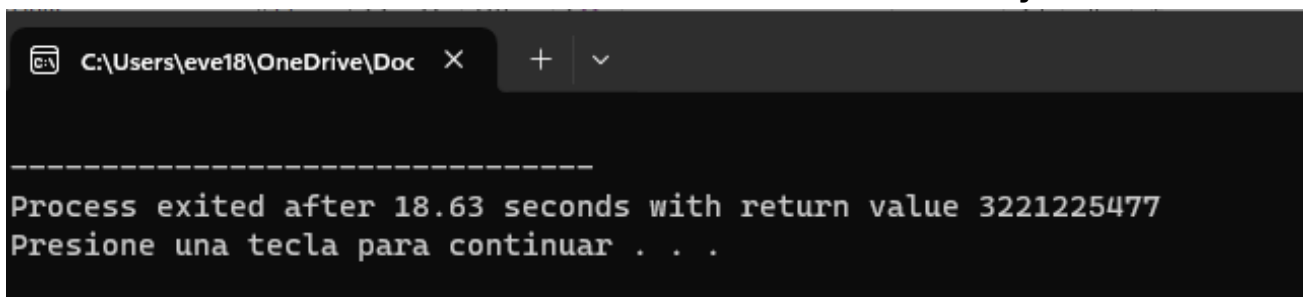


Si se dejan los errores al ejecutar, aparece el siguiente mensaje y la pantalla de ecuación se ve de la siguiente manera:

Mensaje:

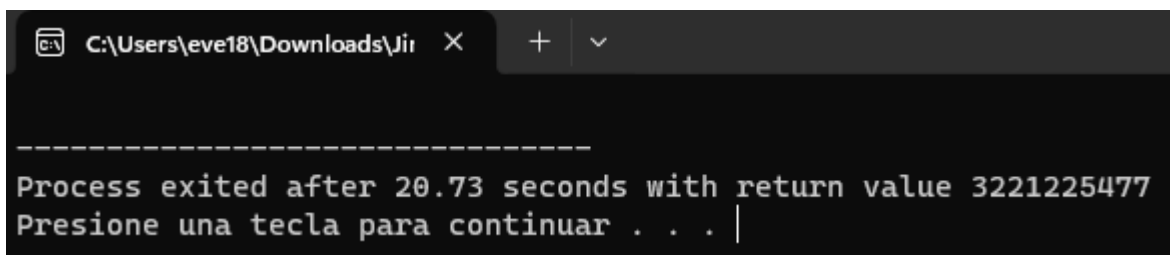
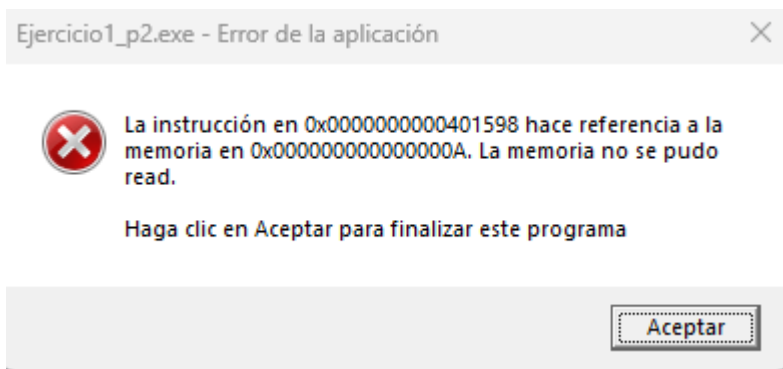


Pantalla de ejecución:



Después a p4 se le asigna el apuntador p1 pero igual hay un error.

Ejecución:



Línea 9: No se le debe agregar el operador (*) antes de p1

Error:

```
9 |
```

```
p4=*p1;
```

Corregido:

```
9 | p4=p1;
```

Ubicación en memoria de los apuntadores y sus variables asociadas

...	...
100	w = 10
103 104	x = 20
107 108	y = 30
111 112	z = 40
115 ...	
800	p1 = 100
803 804	p2 = 104
807	p3 = 108
810 811	p4 = 100
815	

Luego por medio de los apuntadores y los valores que tienen asignados se hacen operaciones y se imprimen los resultados.

Igual hay errores.

Línea 10: En “r1 = p1 + *p2” al apuntador p1 se le debe agregar el operador (*) antes de él, para así poder utilizarlo

Línea 11: En “r2 = p3 * p4” igual se le deben agregar los operadores (*) antes de p3 y p4.

Línea 12: Y en “*r3= *p2+ *p3” se le debe quitar el operador (*) que esta antes de r3 ya que r3 no es un apuntador

Errores:

```
9      p4=p1;  
10     r1= p1 + *p2;  
11     r2= p3 * p4;  
12     *r3= *p2 + *p3;
```

Corregido

```
9      p4=p1;  
10     r1= *p1 + *p2;  
11     r2= *p3 * *p4;  
12     r3= *p2+ *p3;
```

El avance que se logro en este ejercicio fue de un 100%, se analizaron los errores y fueron corregidos.

Para resolver este problema se necesitan de elementos teóricos como: declaración de apuntadores, el correcto uso de los operadores (*) y (&) y de los apuntadores.

La estrategia para resolver este problema fue analizar cada línea del código, después corregir los errores, posteriormente compilar y ejecutar.

En este programa se usan apuntadores, que después se asocian a las variables utilizando el operador (&), y después se realizan operaciones aritméticas utilizando los valores apuntados por los apuntadores.

Implementación:

$r1 = 10 + 20 = 30$

$r2 = 30 * 10 = 300$

$r3 = 20 + 30 = 50$

```
China37:Desktop estudiante$ ./Ejercicio1_p2  
Los resultados son: 30, 300 y 50
```

Ejercicio 2

Codifica, compila y ejecuta el siguiente programa. Explica a grandes rasgos las instrucciones que contiene y qué es lo que hace

El código es un programa el cual utiliza una función llamada “azul” que utiliza paso por referencia por medio de apuntadores.

Primero se declara la función azul que tiene como parámetros dos apuntadores enteros, después en la función main se declaran tres variables de tipos entero (a, b y c) las cuales se inicializan las variables $a = 4$ y $b = 7$, después se manda a llamar la función azul pasando las direcciones de memoria de a y b como argumentos y se asigna el valor retornado a la variable c. Se imprimen los valores de a, b y c.

En la función se tienen como parámetros los apuntadores k y m que toman inicialmente los valores de a y b. Después se hacen operaciones aritméticas con esos apuntadores.

Línea 13: $*k = 7 + 7 = 14 = a$. Ya que el apuntador m toma el valor de b (7) y lo suma, ahora el apuntador k y la variable son iguales a 14.

Línea 14: $*m = 14 + 14 = 28 = b$. Ahora la operación se hace con el nuevo valor de k. Ahora m y b son iguales a 28.

Línea 15: $*k = 28 - 14 = 14 = a$. La operación se hace con los nuevos valores de k y m. k y a ahora toman el valor de 28.

Línea 16: $*m = 14 - 28 = -14 = b$. Pasa lo mismo que en la líneas anteriores. m y b tienen el valor de -14

Línea 17: $2 * -14 = -28$. Se multiplica por 2 el valor de m (-14) y lo retorna.

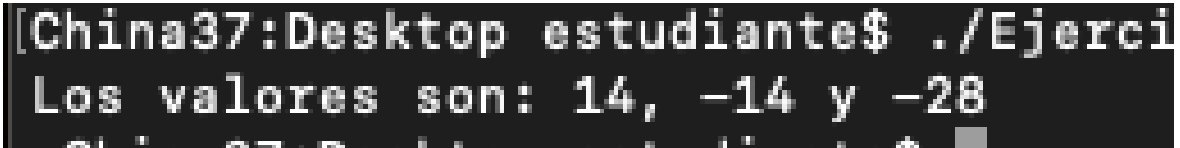
```
11  int azul(int *k, int *m)
12  {
13      *k = *m + *m;
14      *m = *k + *k;
15      *k = *m - *k;
16      *m = *k - *m;
17      return 2 * *m;
18  }
```

Como a c se le asigna el valor retornado en la función azul, c toma el valor de -28.

Y a y b ahora tienen los nuevos valores que obtuvieron en la función azul. Posteriormente se imprimen.

```
8 | c = azul(&a,&b);  
9 | printf("Los valores son: %d, %d y %d \n ",a,b,c);
```

Implementación:



```
[China37:Desktop estudiante$ ./Ejerci  
Los valores son: 14, -14 y -28
```

El avance que se logró en este ejercicio fue de un 100%, se código, compilo y ejecuto el código y se explicó la función de este

Para resolver este problema se necesitan de elementos teóricos como: que es una función y como usarla, sus parámetros, como se hace el paso por referencia cuando se utilizan apuntadores en una función y el retorno

La estrategia para resolver este problema fue analizar cada línea del código, revisar si no tenia errores, al ver que no posteriormente codifique, compile y ejecute y posteriormente analice la salida, el porque de esa salida y cómo funcionaba el código.

En este programa se usan apuntadores para permitir que la función azul acceda y modifique los valores de las variables a y b que se pasan como argumentos, desde la función main, es decir se utilice el paso por referencia con apuntadores.

Ejercicio 3

Codifica y compila el siguiente programa.

a) Completa el programa para llenar correctamente el arreglo

Para poder llenar correctamente el arreglo ponemos el primer valor del arreglo en el for externo y luego los demás valores en los for internos.

<pre>10 11 12 13 14</pre>	<pre>int arr1[4][5][2], var,i,j,k; for (i=0; i<4; i++) { for (j=0; j< 5; j++) { for (k=0; k<2; k++)</pre>
---	---

En el código primero se declara un arreglo tridimensional de tipo entero y 4 variables (var, i, j y k), después se declara un apuntador de tipo entero (point).

En la función main asociamos el apuntador con el arreglo y la asignamos a la variable “var” el valor de 5.

Posteriormente con se utilizan 3 for para recorrer los elementos del arreglo arr1. Estos for inician en 0 y finalizan en el último índice. En cada iteración del ciclo se asigna el valor de var al elemento actual del arreglo y se le incrementa a var 5 para la próxima iteración

Después de inicializar arr1, se calculan los valores de a, b, c y d utilizando aritmética de punteros:

- a se inicializa con el valor del elemento en el lugar 3 del arreglo arr1.
- b se inicializa con el valor del elemento en el lugar 12 del arreglo arr1.
- c se inicializa con el valor del elemento en el lugar 27 del arreglo arr1.
- d se inicializa con el valor del elemento en el lugar 37 del arreglo arr1.

<pre>20 21 22 23</pre>	<pre>int a= *(point+3); int b= *(point+12); int c= *(point+27); int d= *(point+37);</pre>
------------------------------------	---

Se imprime en pantalla los valores de a, b, c y d.

```
24 | printf("a = %d, b= %d, c=%d, d=%d",a,b,c,d);
```

b) Dibuja el arreglo y especifica cuáles son los índices que hacen referencia a las variables a - d. posteriormente agrega impresión de pantalla de dichas variables para validar tu análisis.

		0	1			0	1
	0	5	10		0	55	60
	1	15	20		1	65	70
0	2	25	30	1	2	75	80
	3	35	40		3	85	90
	4	45	50		4	95	100

		0	1			0	1
	0	105	110		0	155	160
	1	115	120		1	165	170
2	2	125	130	3	2	175	180
	3	135	140		3	185	190
	4	145	150		4	195	200

```

C:\Users\eve18\OneDrive\Doc X + v
a = 20, b= 65, c=140, d=190
-----
Process exited after 0.1934 seconds with return value 27
Presione una tecla para continuar . . .

```

c) Utilizando el apuntador y un solo ciclo, recorre el arreglo de tal manera que se modifiquen todos sus valores de la siguiente manera: plano 0, valores múltiplos de 3 (3,6,9,12); plano 1: valores múltiplos de 6 (6,12,18,...), plano 2, valores múltiplos de 7 (7,14,21,28...) plano 3, valores múltiplos de 4 (4,8,12,16)

Primero declaro cuatro variables y las inicializo en 1 (var, var1, var2, y var3)

Utilice un solo ciclo for que finaliza en el último índice de todo el arreglo ya que $4*5*2 = 40$.

```
11 | for (i = 0; i < 40; i++)
```

Después se utilizan 4 if:

Líneas 13 – 18: El primer if toma como condición que si el contador “i” es menor igual a 9 (el ultimo indice del plano 0), se suma el lugar actual del contador mas

el primer elemento del arreglo, después en ese lugar se multiplicara la variable “var” por 3. Luego la variable var se incrementa en 1.

```
13 |  
14 |  
15 |  
16 |  
17 |  
18 |
```

```
if (i <= 9)  
{  
    *(point + i) = var* 3;  
    printf("\t %d", *(point + i));  
    var++;  
}
```

Plano 0:

		0	1
0	0	3	6
	1	9	12
	2	15	18
	3	21	24
	4	27	30

Líneas 19 - 24: El segundo if toma como condiciones que si el contador “i” es mayor a 9 (el primer índice del plano1) y que sea menor igual a 19 (el ultimo indice del plano 1), se suma el lugar actual del contador más el primer elemento del arreglo, después en ese lugar se multiplicara la variable “var1” por 6. Luego la variable var1 se incrementa en 1.

```

19
20
21
22
23
24

```

```

else if ((i>9)&&(i <= 19))
{
    *(point + i) = var1* 6;
    printf("\t %d", *(point + i));
    var1++;
}

```

Plano 1:

		0	1
1	0	6	12
	1	18	24
	2	30	36
	3	42	48
	4	54	60

Líneas 25 - 30: El tercer if toma como condiciones que si el contador “i” es mayor a 19 (el primer índice del plano 2) y que sea menor igual a 29 (el ultimo indice del plano 2), se suma el lugar actual del contador más el primer elemento del arreglo, después en ese lugar se multiplicara la variable “var2” por 7. Luego la variable var2 se incrementa en 1.


```

25
26
27
28
29
30
else if ((i>19)&&(i <= 29))
{
    *(point + i) = var2* 7;
    printf("\t %d", *(point + i));
    var2++;
}

```

Plano 2:

	0	1
0	7	14
1	21	28
2	35	42
3	49	56
4	63	70

Líneas 31 - 36: El ultimo if toma como condiciones que si el contador "i" es mayor a 29 (el primer índice del plano 3) y que sea menor igual a 29 (el ultimo índice del plano 3), se suma el lugar actual del contador más el primer elemento del arreglo, después en ese lugar se multiplicara la variable "var3" por 4. Luego la variable var3 se incrementa en 1.

```

31
32
33
34
35
36
else if ((i>29)&&(i <= 39))
{
    *(point + i) = var3* 4;
    printf("\t %d ", *(point + i));
    var3++;
}

```

Plano 3:

	0	1
0	4	8
1	12	16
2	20	24
3	28	32
4	36	40

El avance que se logró en este ejercicio fue de un 100%, se realizó el código y se completó conforme se indicaba

Para resolver este problema se necesitan de elementos teóricos como: apuntador, como utilizarlo, la aritmética de apuntadores en arreglos, arreglos y ciclo for.

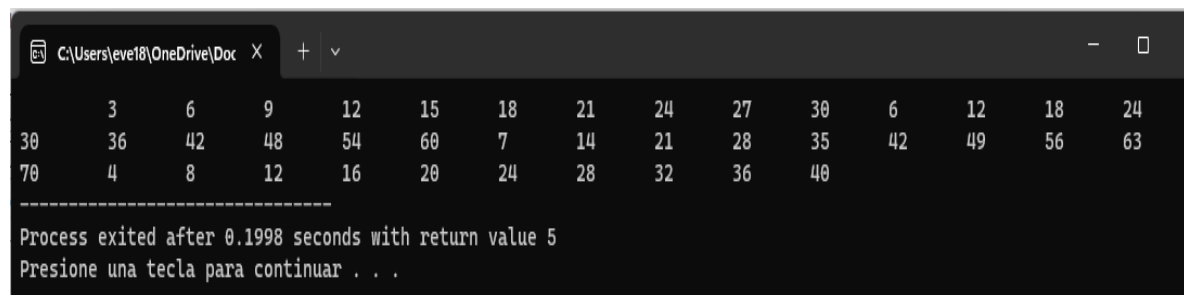
La estrategia para resolver este problema fue solo analizar que tamaño tenía el arreglo para así completar los for. En el inciso c solo analice como podía multiplicar en cada plano por lo que decidí utilizar el if para que solo se utilizara cierto plano y

ahí realizar las instrucciones, si se me complico el pensar como hacerlo, pero realizando varias pruebas lo logre.

En este programa se utiliza la aritmética de apunadores para poder imprimir cierto elemento de un arreglo como sucedió al imprimir las variables a, b, c y d que eran los elementos del arreglo arr1.

```
20 | int a= *(point+3);
21 | int b= *(point+12);
22 | int c= *(point+27);
23 | int d= *(point+37);      *(point + i) = var* 3;
```

Implementación:



```
C:\Users\eve18\OneDrive\Doc X + v
3      6      9      12     15     18     21     24     27     30     6      12     18     24
36     42     48     54     60     7      14     21     28     35     42     49     56     63
70     4      8      12     16     20     24     28     32     36     40

-----
Process exited after 0.1998 seconds with return value 5
Presione una tecla para continuar . . .
```

Ejercicio 4.

Escribe un programa en el cual, se soliciten al usuario 2 valores (almacenadas en 2 variables) y mediante una función que no devuelva ningún valor y que utilice paso por referencia, se calcule lo siguiente:

- a) El resultado de elevar el primero a la potencia que indique el segundo
- b) La división del valor obtenido en el inciso a y el primer valor ingresado

El resultado de a y b se debe almacenar en las mismas variables donde se almacenaron los datos ingresados por el usuario

***Nota: la función solicitada no debe contener ninguna instrucción “printf” ni ninguna variable auxiliar, ya que el print solo aparecerá en la función principal para mostrar los resultados**

Lo que hace el programa es pedir dos números al usuario, para posteriormente elevar el primer numero a la potencia del segundo, luego dividir el resultado de la potencia entre el primer número ingresado.

Para esto primero se declara la función cálculos la cual utilizara como parámetros dos apuntadores de tipo float e igual utilizara paso por referencia.

Después en el main se declaran cuatro variables de tipo float (a, b, x y y) dos se utilizarán para pedir al usuario los números (a y b) y las otras dos es para guardar el valor original de las 2 variables anteriores. Después se manda a llamar la función cálculos, pasando las direcciones de memoria de a y b como argumentos. Se imprimen en pantalla los resultados de dos operaciones matemáticas:

a) $x^y = a$, donde x y y son los valores originales ingresados por el usuario, y a es el resultado de elevar el primer número a la potencia del segundo.

b) $a / x = b$, donde a es el resultado de la potencia anterior, x es el primer valor ingresado por el usuario y b es el resultado de la división

```
x=a;
y=b;
calculos(&a, &b);

printf("a) %.2f ^ %.2f = %.2f\n",x,y,a);
printf("b) %.2f / %.2f = %.2f\n", a,x,b);
```

En la función cálculos se pasan como parámetros las variables a y b en forma de apuntadores, después se declara un variable “aux”, la cual almacena el valor original de a.

Línea 27: Se calcula la potencia de elevar el primer valor ingresado por el usuario a la potencia del segundo valor, el resultado se guarda en la variable "a".

Línea 28: Ahora se divide el resultado de la potencia anterior es decir el nuevo valor de la variable "a" entre el valor original de la variable "a" y el resultado se guarda en la variable b.

```
27 | *a=pow(*a, *b);  
28 | *b= *a / aux;
```

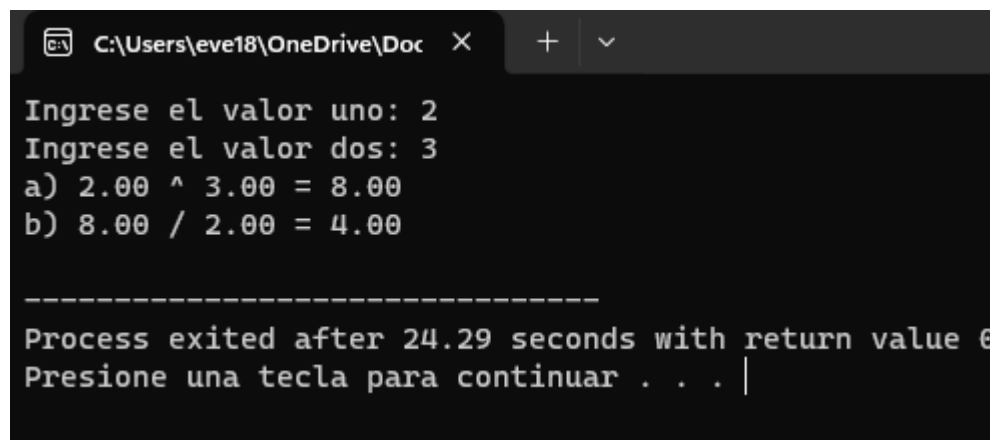
El avance que se logró en este ejercicio fue de un 100%, se realizó el código conforme a las indicaciones que se dieron y la salida fue como se indico

Para resolver este problema se necesitan de elementos teóricos como: que es una función y como usarla, sus parámetros, como se hace el paso por referencia cuando se utilizan apuntadores en una función.

La estrategia para resolver este problema revisar como iba a utilizar el paso por referencia en la función, por lo que me guie de un ejemplo visto en clase para poder realizar el programa. Tuve una dificultad al utilizar el valor original de una variable cuando ya pasa por referencia, pero lo solucioné guardando el valor original en una variable.

En este programa se usan apuntadores para permitir que la función calculos acceda y modifique los valores de las variables a y b que se pasan como argumentos, desde la función main, es decir se utilice el paso por referencia con apuntadores.

Implementación



```
C:\Users\eve18\OneDrive\Doc X + v  
Ingrese el valor uno: 2  
Ingrese el valor dos: 3  
a) 2.00 ^ 3.00 = 8.00  
b) 8.00 / 2.00 = 4.00  
  
-----  
Process exited after 24.29 seconds with return value 0  
Presione una tecla para continuar . . . |
```

Conclusiones

Si se cumplió los objetivos ya que en los diferentes programas se utilizo un apuntador para acceder a las localidad de memoria tanto de datos primitivos como de arreglos. También se utilizaron apuntadores para modificar valores de variables indirectamente como en el ejercicio 2 y 4 que utilizamos apuntadores para acceder a las variables y modificarlas igual se utilizo paso por referencia en funciones en estos ejercicios. En el ejercicio 3 se pudo realiza el manejo correcto de arreglos con apuntadores.

Si contribuyo al aprendizaje ya que todavía me quedaban algunas dudas en el paso por referencia y el como asociar correctamente el apuntador con la variable, al realizar y analizar los ejercicios 1 y 4 me quedo claro como funcionaba el paso por referencia.

Las posibles ventajas es que si en algún momento no quedo claro el tema de apuntadores aquí se pudo reforzar, pero también puede ser una desventaja ya que si no quedo claro y no se entendió el tema de arreglos no se podría realizar el ejercicio 3.

En esta practica no tengo ninguna critica constructiva, todo quedo claro.

El porcentaje global de todos los ejercicios fue del 100% se corrigieron (en caso de tener errores), se cumple cada una de las instrucciones de los requisitos.